

Laboratorium Ilmu Rekayasa dan Komputasi

TEKNIK INFORMATIKA
STEI ITB

IF2224 - Formal Language and
Automata Theory

PASCAL-S Compiler

Milestone 2 - Syntax Analysis

Milestone 2

Syntax Analysis

Release : Selasa, 28 Oktober 2025

Deadline : Minggu, 16 November 2025 pukul 23.59 WIB

Revisi : 3

Daftar Revisi

[4-11-2025] Revisi 1. Perubahan Contoh Hasil Pemrosesan.

[6-11-2025] Revisi 2. Penggabungan node function dan procedure call

[11-11-2025] Revisi 3. Pengubahan parentheses pada node function/procedure-call menjadi wajib

[14-11-2025] Revisi 4. Setelah beberapa pertimbangan, semua fungsi dan prosedur (termasuk writeln) dianggap sebagai IDENTIFIER. Namun, karena revisi cukup mepet deadline, maka perubahan tidak wajib diimplementasikan untuk milestone ini, tetapi harus sudah diimplementasikan untuk milestone 3 nanti. Kami minta maaf atas ketidaknyamanannya.

Daftar Isi

Daftar Revisi.....	2
Daftar Isi.....	4
I. Pendahuluan.....	5
II. Teori Singkat.....	6
III. Spesifikasi Tugas Besar.....	9
IV. Teknis Pengerjaan.....	18
V. Penilaian.....	20
VI. Pengumpulan.....	21
Referensi.....	22
Lampiran A. Link Penting.....	23
Lampiran B. Syntax Diagram.....	24

I. Pendahuluan



Well done, Gus Fring said

Proses anda mengerjakan lexer berhasil dengan baik. Gus Fring mengapresiasi pekerjaan anda. Rupanya, banyak perusahaan yang dimiliki oleh perusahaan induk Los Pollos Hermanos yaitu Madrigal menggunakan layanan lexer ini untuk melaksanakan proses bisnisnya. Madrigal memberikan penghargaan ke Gus Fring dan *funding* lebih untuk Los Pollos Hermanos. Seketika itu juga, lexer buatan Los Pollos Hermanos menjadi program komputer yang sangat *hot* di kalangan *enterprise*. Gus Fring dan Madrigal bersama merintis perusahaan bersama para pekerja barunya (kalian) yang bernama **Tecnicidad**. Gus Fring siap untuk masuk ke tahap berikutnya dari proyek yang kalian kerjakan. Namun, terdapat grup orang yang memiliki ide lain pada proyek kalian ini.



A Wild Buyer appeared, Meet Kendall Roy

Waystar Royco telah bergabung dengan Gojo setelah beberapa kejadian yang terjadi pada perusahaan tersebut. Setelah kejadian tersebut, Kendall Roy tidak memiliki apa-apa kecuali uang yang banyak. Layaknya orang kaya, uang sebanyak apapun tidak akan cukup untuk memenuhi kebutuhannya. Dia selalu menginginkan menggantikan ayahnya yang sudah tiada untuk memimpin Waystar tetapi hal tersebut tidak kunjung terjadi dan malahan posisi tersebut diambil orang lain

Beberapa tahun kemudian, Kendall memutuskan untuk kembali ke permainan politik bisnis dan memulai dari 0 (lebih ke memulai dari 1 miliar dolar sih but whatever). Tidak ada angin dan tidak ada hujan, dia membeli *share* untuk Madrigal sebesar 32%. Kendall menjadi biggest *shareholder* dari Madrigal dan ingin mengedepankan teknologi lebih dari segalanya. Kendall bertemu dengan Gus Fring selaku PO dari proyek Pascal-S compiler ini dan menyimpulkan bahwa proyek ini akan membawa Madrigal ke era kejayaan. Gus mengira Kendall ini lebay tapi dia biarkan saja. Pengembangan tahap selanjutnya yaitu Syntax Analysis seharusnya dimulai bulan depan tetapi Kendall mau proyek ini dimulai dengan cepat. Jadi selamat datang *developer* baru Tecnicidad dan selamat mengerjakan proyek milestone 2 yang tanggal mulainya dimajukan Kendall :))))).

II. Teori Singkat

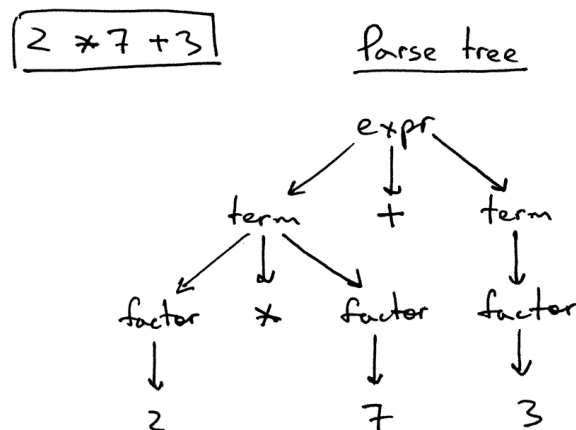
Pada milestone I lexical analysis, source code sudah dibentuk menjadi token-token tertentu. Token-token ini akan digunakan oleh tahap selanjutnya yaitu Syntax Analysis atau sering disebut parser.

Syntax Analysis atau **Parser** bertugas untuk memeriksa urutan token yang dihasilkan oleh lexical analyzer agar sesuai dengan **aturan tata bahasa (grammar)** dari bahasa pemrograman yang dikompilasi.

Parser akan membangun struktur sintaks seperti **Parse Tree** yang merepresentasikan struktur hierarkis program sumber berdasarkan grammar yang telah didefinisikan.

Tahapan Parser bertujuan untuk:

- Memastikan urutan token membentuk **struktur sintaks yang valid**.
- Mendeteksi dan melaporkan **kesalahan sintaks** (syntax error).
- Mempersiapkan representasi program untuk tahap berikutnya.



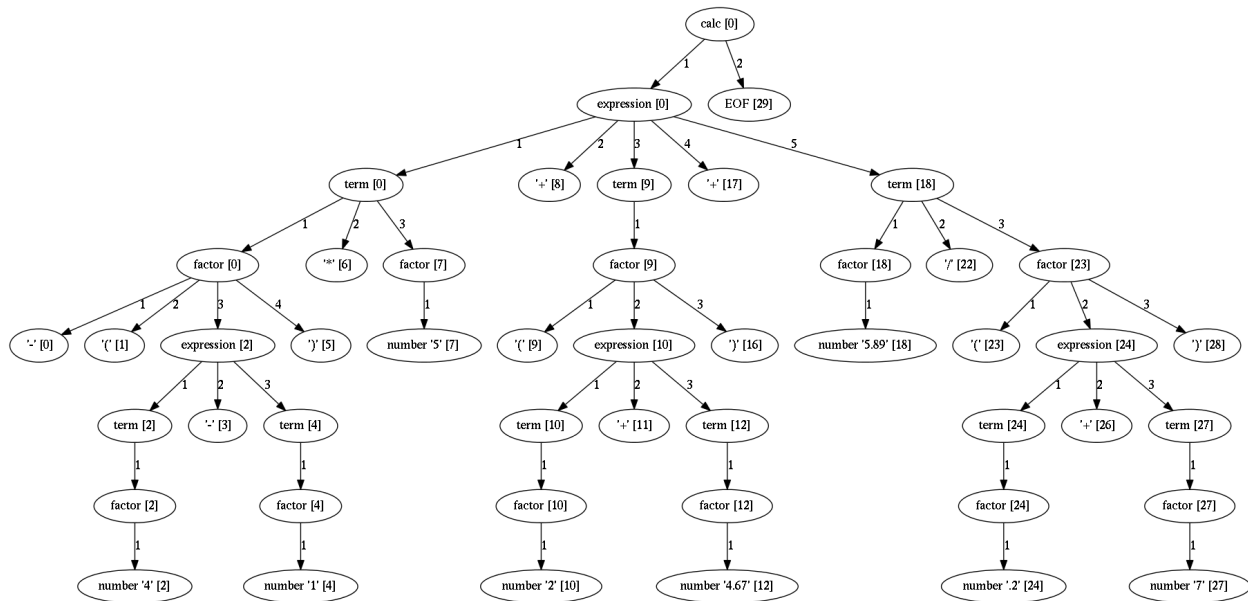
Gambar Parse Tree

(Sumber: <https://ruslanspivak.com/lbasi-part7>)

Parse Tree

Parse Tree adalah representasi lengkap dari struktur sintaks program sesuai dengan grammar bahasa yang digunakan. Setiap node di dalam parse tree

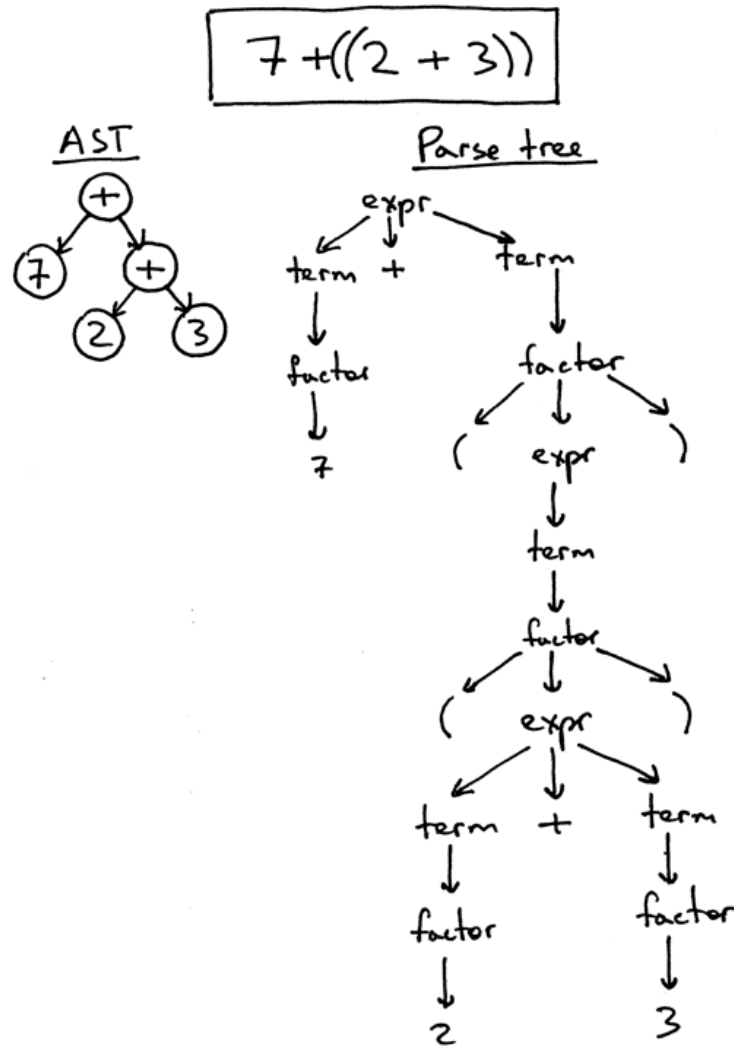
merepresentasikan **produksi grammar**, sehingga pohon ini memuat semua simbol terminal dan non-terminal.



Contoh Parse Tree

Sumber: https://textx.github.io/Arpeggio/latest/images/calc_parse_tree.dot.png

Parse tree berguna untuk menunjukkan bagaimana token-token hasil lexical analysis membentuk struktur program yang sesuai dengan aturan grammar.



Gambar Contoh Pembuatan Parse Tree dan Perbedaannya dengan AST (Abstract Syntax Tree)
(Sumber: <https://ruslanspivak.com/lbasi-part7>)

III. Spesifikasi Tugas Besar

Terdapat beberapa penyesuaian pada DFA yang dibuat sebelumnya untuk pengerjaan kedepannya, yaitu beberapa kata Bahasa Inggris diubah menjadi Bahasa Indonesia dengan padanan berikut.

Kata Bahasa Inggris	Kata Bahasa Indonesia
and	dan
array	larik
begin	mulai
case	kasus
const	konstanta
div	bagi
do	lakukan
downto	turun-ke
else	selain-it
end	selesai
for	untuk
function	fungsi
if	jika
mod	mod
not	tidak
of	dari
or	atau
procedure	prosedur
program	program

record	rekaman
repeat	ulangi
then	maka
to	ke
type	tipe
until	sampai
var	variabel
while	selama

Perubahan cukup dilakukan pada file konfigurasi DFA saja. Diagram DFA nya tidak perlu diubah. Lalu untuk pengerjaan kedepannya, input source code dari awal hingga pemrosesan akhirnya akan menggunakan source code PASCAL-S yang kata Bahasa Inggrisnya ditranslasikan menjadi Bahasa Indonesia menurut tabel padanan diatas. Apabila ada kata Bahasa Inggris yang tidak termuat pada tabel diatas, anggap saja **kata tersebut tidak perlu ditranslasikan**.

Kemudian pada milestone ini, Anda diminta untuk membuat **tahapan kedua dari compiler**, yaitu **syntax analysis**. Parser berfungsi melakukan analisis sintaksis (syntax analysis) dengan menggunakan **Recursive Descent** untuk mengenali struktur gramatikal dalam deretan token.

Tahapan ini bertujuan untuk menganalisis struktur sintaksis dari list of tokens yang dihasilkan lexer dan membangun **Parse Tree** yang merepresentasikan hierarki struktur program Pascal-S sesuai dengan grammar bahasa. Parse Tree ini akan menjadi masukan bagi tahap semantic analysis dalam proses kompilasi.

Masukan	List Token (misalnya VAR(X), ASSIGN(=), OPERATOR(+))
Keluaran	Parse Tree
Algoritma	Recursive Descent

Berikut merupakan daftar node yang mungkin muncul pada hasil Parse Tree.

No	Nama Node	Deskripsi	Aturan Produksi	Contoh
1	<program>	Node root yang merepresentasikan keseluruhan program Pascal-S	program $\rightarrow$ program-header + declaration-part + compound-statement + DOT	Seluruh struktur program dari awal hingga akhir
2	<program-header>	Bagian kepala program yang berisi nama program	KEYWORD(program) + IDENTIFIER + SEMICOLON	program Hello;
3	<declaration-part>	Bagian deklarasi yang dapat berisi const, type, var, procedure, function	(const-declaration)* + (type-declaration)* + (var-declaration)* + (subprogram-declaration)*	Semua deklarasi sebelum "mulai"
4	<const-declaration>	Deklarasi konstanta	KEYWORD(konstanta) + (IDENTIFIER = value + SEMICOLON)+	konstanta MAX = 100;
5	<type-declaration>	Deklarasi tipe data baru	KEYWORD(tipe) + (IDENTIFIER = type-definition + SEMICOLON)+	tipe Range = 1..10;
6	<var-declaration>	Deklarasi variabel	KEYWORD(variabel) + (identifier-list + COLON + type + SEMICOLON)+	variabel a, b: integer;
7	<identifier-list>	Daftar identifer	IDENTIFIER	a, b, c

	ist>	yang dipisahkan koma	(COMMA + IDENTIFIER)*	
8	<type>	Tipe data (integer, real, boolean, char, array)	KEYWORD(integer /real/boolean/char) atau array-type	integer, larik[1..10] dari integer
9	<array-type>	Definisi tipe array	KEYWORD(larik) + LBRACKET + range + RBRACKET + KEYWORD(dari) + type	larik[1..10] dari integer
10	<range>	Rentang nilai untuk array atau subrange	expression + RANGE_OPERATOR(..) + expression	1..10, 'a'..'z'
11	<subprogram-declaration>	Deklarasi prosedur atau fungsi	procedure-declaration atau function-declaration	prosedur print(x: integer);
12	<procedure-declaration>	Deklarasi prosedur	KEYWORD(prosedur) + IDENTIFIER + (formal-parameter-list)? + SEMICOLON + block + SEMICOLON	Prosedur dengan parameter opsional
13	<function-declaration>	Deklarasi fungsi	KEYWORD(fungsi) + IDENTIFIER + (formal-parameter-list)? + COLON + type + SEMICOLON + block + SEMICOLON	Fungsi yang mengembalikan nilai
14	<formal-param	Daftar parameter	LPARENTHESIS +	(x, y:

	eter-list>	formal	parameter-group (SEMICOLON + parameter-group) * + RPARENTHESIS	integer; z: real)
15	<compound-statement>	Blok statement yang diawali begin dan diakhiri end	KEYWORD(mulai) + statement-list + KEYWORD(selesai)	mulai ... selesai
16	<statement-list>	Daftar statement yang dipisahkan semicolon	statement (SEMICOLON + statement)*	Urutan statement dalam blok
17	<assignment-statement>	Statement penugasan nilai ke variabel	IDENTIFIER + ASSIGN_OPERATOR(:=) + expression	x := 5, y := x + 10
18	<if-statement>	Statement kondisional	KEYWORD(jika) + expression + KEYWORD(maka) + statement + (KEYWORD(selain-itu) + statement)?	jika x > 0 maka y := 1 selain-itu y := 0
19	<while-statement>	Perulangan dengan kondisi di awal	KEYWORD(selama) + expression + KEYWORD(lakukan) + statement	selama x < 10 lakukan x := x + 1
20	<for-statement>	Perulangan dengan counter	KEYWORD(untuk) + IDENTIFIER + ASSIGN_OPERATOR + expression + (KEYWORD(ke)/KEYWORD(turun-ke)) + expression + KEYWORD(lakukan) + statement	untuk i := 1 ke 10 lakukan ...

21	<code><procedure/function-call></code>	Pemanggilan prosedur atau fungsi	IDENTIFIER + (LPARENTHESIS + parameter-list + RPARENTHESIS)?	<code>writeln('Hello'), print(x, y)</code>
22	<code><parameter-list></code>	Daftar parameter aktual saat pemanggilan	expression (COMMA + expression)*	<code>'Result = ', b, 100</code>
23	<code><expression></code>	Ekspresi yang menghasilkan nilai	simple-expression (relational-operator + simple-expression)?	<code>x + y, a > b, 5 * (x + 1)</code>
24	<code><simple-expression></code>	Ekspresi tanpa operator relasional	(ARITHMETIC_OPERATOR(+/-))? term (additive-operator + term)*	<code>x + y - z, 5 * 2</code>
25	<code><term></code>	Bagian ekspresi dengan prioritas lebih tinggi	factor (multiplicative-operator + factor)*	<code>x * y, a bagi b</code>
26	<code><factor></code>	Unit terkecil dalam ekspresi	IDENTIFIER / NUMBER / CHAR_LITERAL / STRING_LITERAL / (LPARENTHESIS + expression + RPARENTHESIS) / LOGICAL_OPERATOR(tidak) + factor / function-call	<code>x, 5, 'a', (x+y), flag "tidak"</code>
27	<code><function-call></code>	Pemanggilan fungsi yang mengembalikan nilai	IDENTIFIER + LPARENTHESIS + (parameter-list)? + RPARENTHESIS	<code>sqrt(16), max(a, b)</code>

28	<relational-operator>	Operator perbandingan	=, <, <=, >, >=	x = 5, y > 10
29	<additive-operator>	Operator penjumlahan/pengurangan	+, -, atau	x + y, a atau b
30	<multiplicative-operator>	Operator perkalian/pembagian	*, /, bagi, mod, dan	x * y, a bagi b, p dan q

Program harus menggunakan Recursive Descent dengan **grammar yang di-hardcode dalam program** dan keseluruhan grammar dijelaskan kembali dalam laporan yang dibuat.

Input awal dari program **tetap merupakan source code PASCAL-S** sehingga sebelum dimasukkan ke parser, kode dimasukkan terlebih dahulu ke lexer.

Input Command (contoh yang digunakan menggunakan Python) Format: python [Compiler] [Kode Pascal]
python compiler.py program.pas
Isi program.pas
<pre> program Hello; variabel a, b: integer; mulai a := 5; b := a + 10; writeln('Result = ', b); selesai.</pre>
program.pas ketika diubah menjadi token
<pre> KEYWORD(program) IDENTIFIER(Hello) SEMICOLON(;) KEYWORD(variabel) IDENTIFIER(a) COMMA(,)</pre>

```

IDENTIFIER(b)
COLON(:)
KEYWORD(integer)
SEMICOLON(;)
KEYWORD(mulai)
IDENTIFIER(a)
ASSIGN_OPERATOR(:=)
NUMBER(5)
SEMICOLON(;)
IDENTIFIER(b)
ASSIGN_OPERATOR(:=)
IDENTIFIER(a)
ARITHMETIC_OPERATOR(+)
NUMBER(10)
SEMICOLON(;)
KEYWORD(writeln)
LPARENTHESIS((
STRING_LITERAL('Result = ')
COMMA(,)
IDENTIFIER(b)
RPARENTHESIS())
SEMICOLON(;)
KEYWORD(selesai)
DOT(.)

```

Proses (grammar belum tentu benar)

```

program → program-header declaration-part compound-statement DOT
program-header → KEYWORD(program) IDENTIFIER SEMICOLON
declaration-part → ...
compound-statement → ...
... grammar lainnya

```

Berdasarkan grammar "program", non-terminal yang pertama ditemukan adalah "program-header". Setelah itu di dalam "program-header" dilakukan pengecekan token input.

```

KEYWORD(program)
IDENTIFIER>Hello)
SEMICOLON(;)

```

Karena token input sesuai, berarti "program-header" valid. Sehingga output sementara akan seperti ini

```

<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER>Hello)
│   └── SEMICOLON(;)
... sisa output

```

... sisa output

Kemudian sisa pengecekan dilakukan untuk "declaration-part", "compound-statement", dan DOT.

Kemudian juga disini dilakukan **error checking** di level parser. Contohnya adalah misal untuk "program-header", token yang dibaca adalah seperti berikut.

```

KEYWORD(program)
IDENTIFIER>Hello)
DOT(.)

```

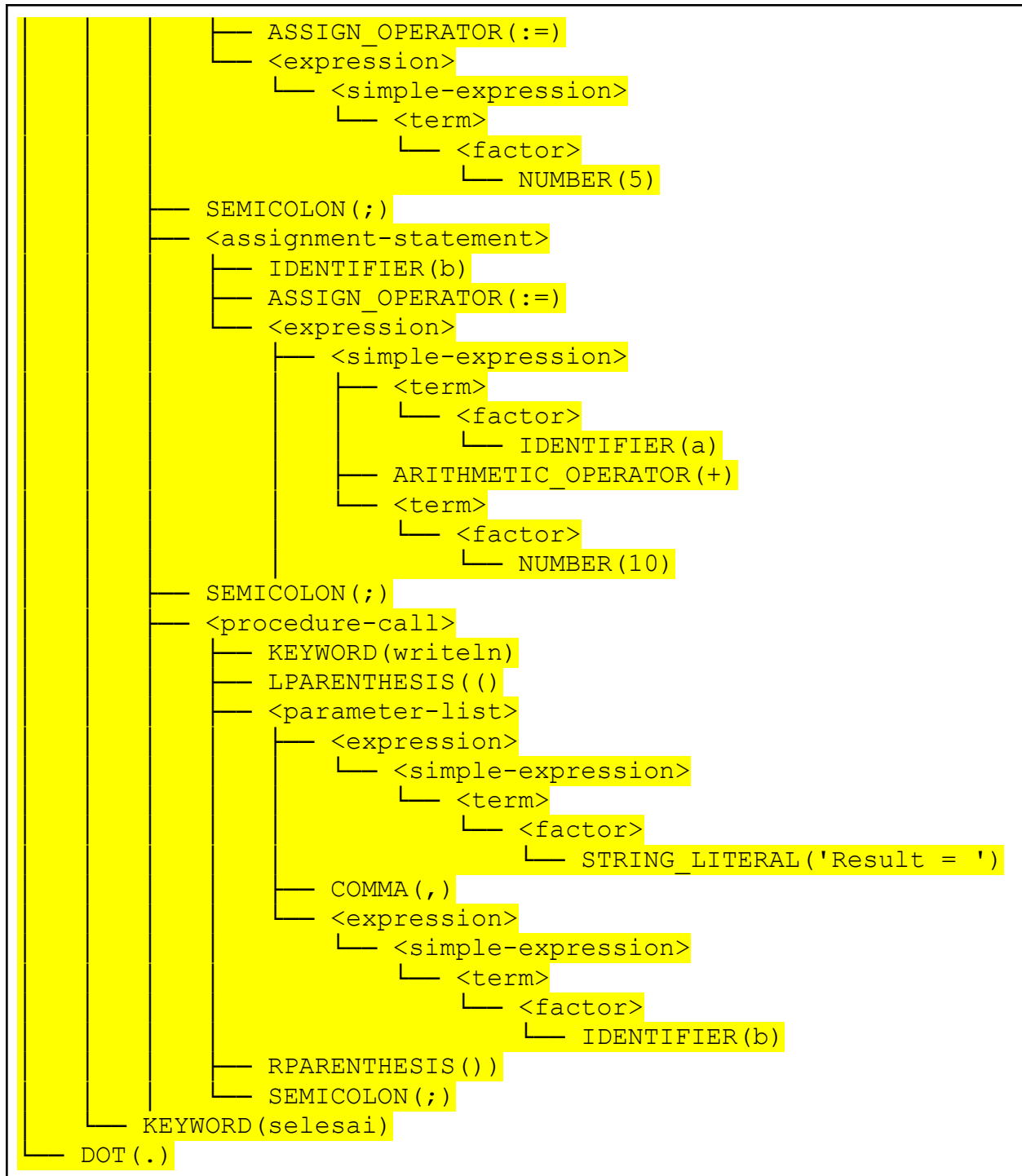
Maka parser akan mengeluarkan error. Untuk pesan error dibebaskan yang penting informatif. Contohnya adalah "Syntax error: unexpected token DOT(.), expected SEMICOLON(;)".

Keluaran (output)

```

<program>
├── <program-header>
│   ├── KEYWORD(program)
│   ├── IDENTIFIER>Hello)
│   └── SEMICOLON(;)
├── <declaration-part>
│   └── <var-declaration>
│       ├── KEYWORD(variabel)
│       ├── <identifier-list>
│       │   ├── IDENTIFIER(a)
│       │   ├── COMMA(,)
│       │   └── IDENTIFIER(b)
│       ├── COLON(:)
│       ├── <type>
│       │   └── KEYWORD(integer)
│       └── SEMICOLON(;)
├── <compound-statement>
│   ├── KEYWORD(mulai)
│   ├── <statement-list>
│   │   └── <assignment-statement>
│   │       └── IDENTIFIER(a)

```



Untuk diagram syntax dari PASCAL-S dapat dilihat pada bagian [lampiran](#) atau dapat dilihat langsung pada paper di bagian [referensi](#).

IV. Teknis Pengerjaan

- Bahasa pemrograman yang digunakan **disamakan** dengan pengerjaan milestone sebelumnya.
- Pengerjaan dilakukan pada **repositori yang sama** dengan milestone sebelumnya.
- Anda **WAJIB** membuat **laporan** dengan format **Laporan-X-[KODE].pdf**, dengan **X** adalah **nomor Milestone** dan **Kode** adalah **Kode Kelompok**.
- **Laporan setidaknya berisi:**
 - Cover dengan foto kelompok menggantikan logo ITB.
 - Landasan Teori
 - Jelaskan teori-teori atau konsep yang berhubungan atau digunakan pada Milestone 2.
 - Perancangan & Implementasi
 - Implementasi yang ditambahkan dan berhubungan dengan Milestone 2.
 - Pengujian
 - Setidaknya 5 pengujian kasus unik.
 - Setiap pengujian harus menunjukkan hasil input/output program.
 - Bukti berupa screenshot input/output program.
 - Kesimpulan dan Saran
 - Lampiran berupa:
 - Link Release Repository Github.
 - Pembagian Tugas (menunjukkan persentase kontribusi).
 - **Grammar yang digunakan.**
 - Referensi yang digunakan selama pengerjaan Milestone 2.
- Jika ada pertanyaan lebih lanjut, silahkan ditanyakan pada [QNA](#).

- Link Pembagian Kelompok: [Sheets Pembagian Kelompok](#)
- Link QNA: [Sheets QNA](#)

V. Penilaian

Program (70 poin)			
No	Komponen	Poin	Syarat Poin Maksimal
1.	Implementasi Recursive Descent	10	Algoritma diimplementasikan secara tepat dan dapat memeriksa kebenaran syntax pada input
2.	Kebenaran Parse Tree	12	Parse Tree yang dihasilkan sesuai dengan input token yang diberikan serta grammar Pascal-S
3.	Struktur/modularitas program	8	Kode ditulis dengan struktur modular menggunakan fungsi atau kelas yang terpisah berdasarkan tanggung jawab. Alur eksekusi program mudah diikuti dan tidak redundan
4.	Error handling	5	Program dapat menangani error dengan baik (pesan informatif dan error tidak menyebabkan crash)
5.	Penerimaan input	3	Program dapat membaca file input hasil tokenisasi pada milestone 1 (.txt)
6.	Penghasilan output	2	Program mengeluarkan output Parse Tree yang terformat dengan jelas ke terminal
7.	Dokumentasi/komentar	5	Kode diberi komentar yang informatif dan tidak berlebihan
8.	Aturan Grammar	10	Grammar yang tepat untuk bahasa Pascal-S di hardcode secara terstruktur
9.	Pengujian	15	Program berhasil lolos seluruh kasus uji

Laporan (30 poin)			
No	Komponen	Poin	Syarat Poin Maksimal
1.	Penjelasan konsep	10	Penjelasan mengenai konsep lexer yang mencakup: • Definisi dan peran parser • Keterkaitan dengan lexer • Algoritma Recursive Descent • Parse Tree
2.	Perancangan program	5	Penjelasan mengenai rancangan program yang mencakup teknologi yang digunakan, struktur program, fungsi/kelas utama, dan alur kerja program. Terdapat pula justifikasi atas pilihan implementasi
3.	Hasil implementasi	2	Dokumentasi mengenai hasil implementasi, dapat berupa tangkapan layar dengan penjelasan
4.	Test case	3	Dokumentasi mengenai pengujian mandiri (minimal 5 dan mencakup berbagai kasus)
5.	Aturan Grammar	10	Tuliskan kembali seluruh Grammar yang digunakan secara rapi

VI. Pengumpulan

- **Deliverables:**
 - Laporan
 - Release Repository
- Pada setiap Milestone, Anda **WAJIB** mengumpulkan **Link menuju Release pada Repository Github.**
 - Gunakan format tag berupa **v0.X.Y** dengan **X** adalah **nomor milestone** dan **Y** adalah **nomor pengumpulan**.
 - Sebagai contoh, untuk milestone 1, release pertama, dan pengumpulan pertama adalah **v0.1.1**. Untuk pengumpulan selanjutnya karena revisi, gunakan tag **v0.1.2** dan seterusnya sesuai nomor pengumpulan. Untuk milestone selanjutnya, pengumpulan dimulai kembali dari satu yaitu **v0.2.1, V0.2.2**, dan seterusnya.
- **Laporan** dikumpulkan dengan cara **meletakkan** hasil **PDF** dalam **folder doc** dengan format **Laporan-X-[KODE].pdf**, dimana **X** adalah **nomor milestone**. Pastikan isi laporan sudah sesuai dengan [teknis pengerjaan](#).
- Link Pengumpulan: [Pengumpulan Milestone](#)
- Deadline milestone: **Minggu, 16 November 2025 pukul 23.59 WIB**

Referensi

"PASCAL-S: A Subset and its implementation"

<http://pascal.hansotten.com/uploads/pascals/PASCAL-S%20A%20subset%20and%20its%20Implementation%20012.pdf>

"Let's Build A Simple Interpreter. Part 7: Abstract Syntax Trees"

<https://ruslanspivak.com/lbasi-part7>

"Contoh Gambar Parse Trees"

https://textx.github.io/Arpeggio/latest/images/calc_parse_tree.dot.png

Lampiran A. Link Penting

Laman Dokumen Terpusat: [☰ Dokumen Terpusat - Tubes IF2224 TBFO](#)

Laman Pembagian Kelompok: [📁 Pembagian Kelompok - Tubes IF2224 TBFO](#)

Laman QnA Tugas Besar: [📁 QNA - Tubes IF2224 TBFO](#)

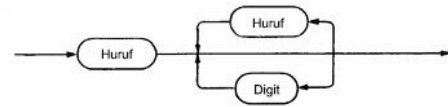
Form Pengumpulan: [Form Pengumpulan - Tubes IF2224 TBFO](#)

Lampiran B. Syntax Diagram

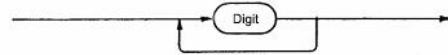
Notes

1. Round boxes denote symbols of the language Pascal, rectangular boxes denote syntactic constructs represented by diagrams.
2. Separators may be inserted between any two symbols. However, no separators must occur within numbers and identifiers.
3. At least one separator must occur between consecutive identifiers, numbers, and word-symbols (such as BEGIN, END).
4. Separators are blanks, ends of lines, and comments. A comment is an arbitrary sequence of characters enclosed within a pair of comment brackets (* and *).
5. The occurrence of the non-qualified word identifier in a syntax diagram implies that at this point an arbitrary, new identifier may be chosen. This identifier thereby becomes a constant-, type-, variable-, field-, function-, or procedure identifier.

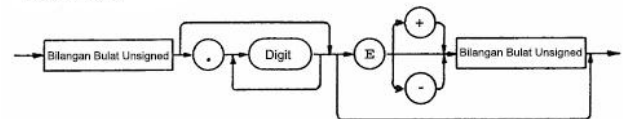
Pengenal



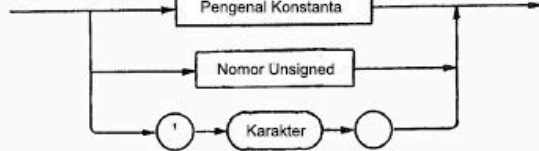
Bilangan Bulat Unsigned



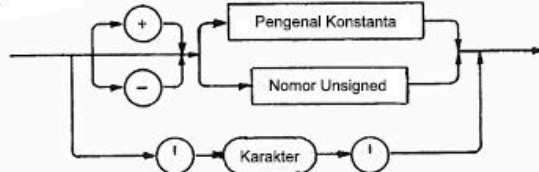
Angka Unsigned



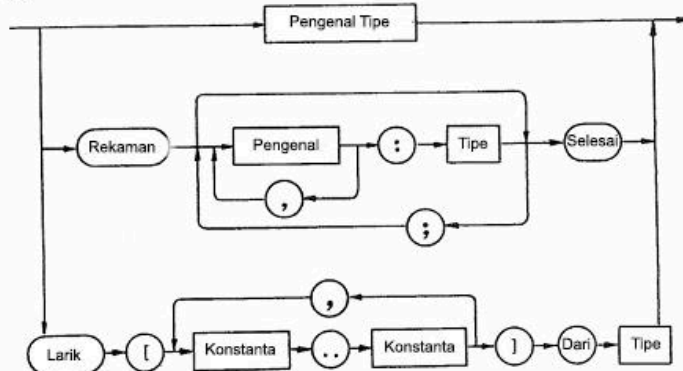
Konstanta Unsigned



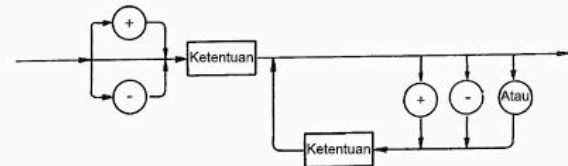
Konstanta



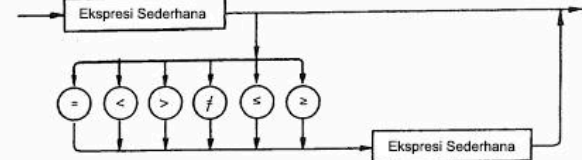
Tipe



Ekspresi Sederhana



Ekspresi



Daftar Parameter

